

Plexus Atlas — jax-morph

Progress note — written to be read, not decoded

31 July 2026

The twin of `plexus2_discovery.pdf`, for the other half of the programme.
Plain English throughout. Organised by phase, because phases are where we stop and you decide.

1. What we are doing — the second way to grow the language

There are two ways to grow the Plexus vocabulary, and as of today both are running.

Discovery (the Okuda folder) searches for a mechanism nobody has written down. It proposes a change to the model's structure, says what it expects *before* running, runs, and records whether it was right. It is the harder science and the slower loop.

Atlas (this folder) is the other one, and it is what Appendix E of the Plexus 2 paper promises. It takes a published system whose code *exists*, reads that code, and asks a narrower question: *can the Plexus operator algebra say everything this framework says?* Whatever the language cannot say becomes a new operator. Whatever it can say twice over is evidence that the language is converging on something real.

The target is jax-morph — Deshpande, Mottes *et al.*, *Engineering morphogenesis of cell clusters with differentiable programming* (Nature Computational Science, 2025). Cells as soft particles that grow, divide, die, push on one another, secrete and read chemical signals, and decide what to do next through a small gene network. Apache licensed, pure Python, 24 mechanisms — and, the point, it runs.

Two deliverables, and neither one alone is enough.

1. **The figure.** One headline result of the paper, reproduced inside Plexus out of operators translated from their code — not fitted to their output.
2. **The ledger.** Every mechanism in the repository sorted into one of three buckets: something the promoted language already has, something it has but must widen, or something genuinely new.

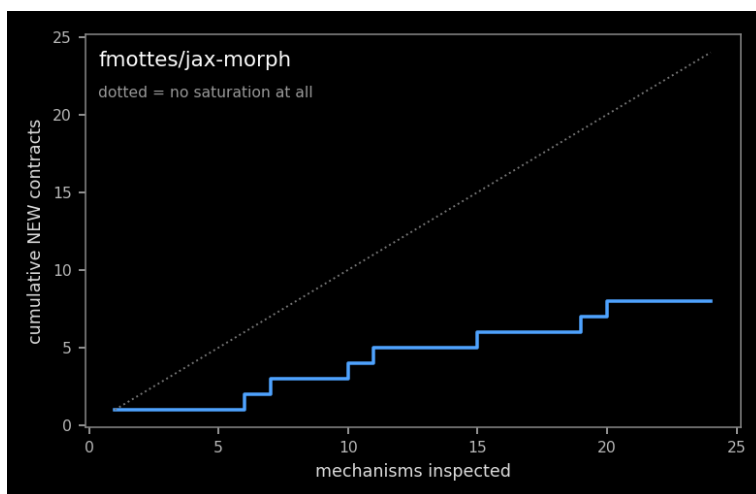
The figure without the ledger is a demo. The ledger without the figure is a spreadsheet nobody has checked against reality.

What we compare against: the promoted language, and nothing else. 52 registered operator contracts in `plexus.operators` — validated, importable, typed, in the engine. Code sitting in `prototype/` or in the `candidates/` anti-chamber is not part of the language and does not enter this measurement. Measuring against code nobody has reviewed would be measuring the wrong thing.

2. Where we actually are

Phases 0 to 3 are done. Every one of the 24 mechanisms has been read at source, normalized into a typed Plexus contract, attacked by a skeptic, and — where it was new or a refinement — implemented as a Plexus operator with a test. Sixteen operator modules exist, **all sixteen import, register the implementation their record claims, agree with their recorded signature, pass a test, and co-exist in one registry.**

The first ledger is in. Eight genuinely new contracts, six further implementations of them, two aliases of things we already had, two refinements, six entries with no biology to extract. And the forward composition RUNS: the authors’ proliferation model, rebuilt from those operators, grows a jammed cluster from 4 cells to 124 — against their 82. That last disagreement is the most useful number the atlas has produced, and §8 is about it.



The measurement, as far as it goes. The dotted line is the world in which every mechanism is new vocabulary; the steps are what actually happened. One repository cannot show saturation — the second and third can.

3. Why a target with code changes everything

The thing Okuda could never give us: an answer to compare against.

Okuda *et al.* published a paper with no code. So when our tube looks wrong there are three explanations and no way to separate them: the operator is wrong, the parameter is wrong, or we read the figure wrong. Days of the discovery campaign went into exactly that fog. Here the authors’ own implementation sits in `papers/jax-morph` and, as of today, *runs* on this machine in its own isolated Python. We can put our reconstruction and their implementation on the same initial condition and subtract. Faithfulness stops being a judgement and becomes a number, against a threshold written down in advance.

There is a second reason, and it is the more interesting one. `jax-morph` is not merely a simulator: it is built from *composable physics steps* that compose by operator splitting, declare which state fields they read and write, and are cross-validated when a model is assembled. That is, independently, most of the Plexus thesis. If two groups arrived at nearly the same decomposition from opposite directions, the decomposition is probably not arbitrary — and the places where they differ are where we should expect to learn something.

Three differences are already visible from their documentation, before any agent has read a line:

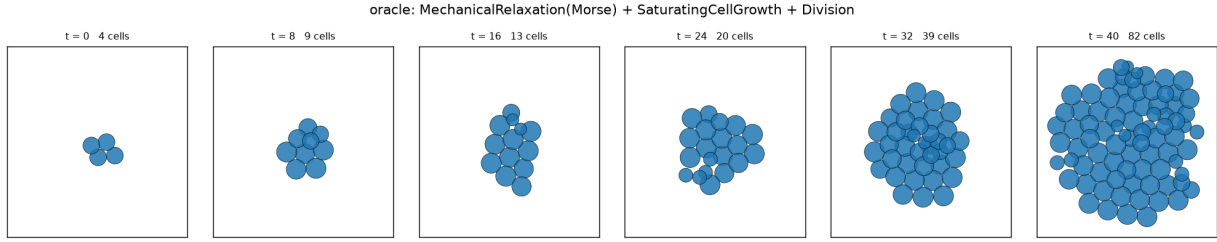
- They classify a step by its **time-scale**: a fast constraint solved at the current configuration, a finite-rate process, or an instantaneous event. Plexus classifies an operator by *which part of the state it touches*. The two are orthogonal, and we do not have theirs.
- Their randomness has a **contract**: a stochastic step records what it sampled, can replay it, and can score it — which is what makes a discrete event like cell division differentiable at all. Plexus has no such contract.
- Their macro-step is an **explicit first-order operator split** with a stated error term. Ours is a schedule.

None of those three is a biological operator. All three are candidates to improve the *language*, which is the second thing the atlas is for.

4. What is genuinely working

Everything here was run today, and its output checked.

piece	state	what was verified
frozen baseline	done	52 registered contracts, 42 canonical and 10 aliases, across 10 families. Written to a file, so the measurement cannot drift while it is being taken.
the oracle	done	jax-morph 0.4.0 installed from our local clone (ace08b8) into its own interpreter. Nothing in the Plexus environment can import it.
oracle determinism	done	the authors' proliferation model, run twice from one seed, is <i>bit-identical</i> . Without this, a later disagreement could be their noise rather than our error.
first reference run	done	4 cells $\rightarrow$ 82 over 40 macro-steps; cluster radius 0.65 $\rightarrow$ 3.23; the division buffer never overflows. Trajectory, summary and figure on disk, each with its provenance.
record + validator	done	twelve rules. Self-tested against a record built to break every one: all twelve fire, and a well-formed record passes in silence.
saturation ledger	done	self-tested, including its right to overrule the record: a “new” whose name is already registered is demoted to <i>alias</i> and the disagreement is reported, not applied quietly.
mechanism inventory	done	24 mechanisms from the clone's syntax tree — 20 classes plus 4 architectural contracts that are not classes and that a scan alone would have missed.
the guarded driver	done	every agent call is a transaction: blast radius, rung, twelve rules, then commit or revert.
one live agent call	done	excavator on Division : committed in 6.3 minutes, nine reimplementer traps, verdict correctly left blank, and an explicit list of what it could <i>not</i> establish.
the paper, readable	done	29 pages, 14,142 words, extracted with page markers and greppable — because the first agent call discovered it could not open the PDF.



The reference, running here. Their model, their code, our machine: mechanical relaxation under a Morse potential, saturating growth, stochastic division. All six panels share one spatial scale — the discovery loop spent a day being fooled by panels that did not.

5. The team of agents — what is switched on

Six roles, each seeing one mechanism at a time.

role	what it does	may edit	budget
excavator	reads the mechanism at source: what it does to the state, the equations, the role of every knob, and what a reimplementer would get wrong. Forbidden from judging novelty.	the record	10 min
normalizer	writes it as a typed Plexus operator and commits to a verdict, with the whole promoted language in front of it.	the record	8 min
skeptic	tries to <i>refute</i> that verdict, in whichever direction it went; defaults to refuted when unsure.	nothing	5 min
implementer	writes the Plexus operator into the anti-chamber, with a test of one property statable without the reference.	code	15 min
differ	writes one metric and one threshold <i>first</i> , then runs both implementations on the same initial condition.	code + record	12 min
curator	promotes, or refuses to. Refusal is a result.	code + docs	10 min

The agents have no authority over the record

This is the part I would ask you to check hardest, because it is the lesson from the other folder. Every discovery-loop error came from a measurement nobody had checked, written down by an agent doing its best. So here the agents decide nothing. They edit; the driver decides whether the edit survives.

1. **Blast radius.** Only the mechanism it was given may have changed. An agent that tidies a neighbouring entry in passing has destroyed our ability to attribute anything, so the whole call is thrown away.
2. **Rung.** Each role can only reach its own status. The excavator cannot declare something validated. A rung is earned by an artefact — a code path that resolves, a module that imports, an oracle run on disk — never by an assertion.
3. **The twelve rules.** If the record does not validate, the edit is reverted and the violations are handed back to the agent, *once*. A second failure marks the mechanism **blocked** and leaves it for a human. It is never silently skipped.

Every revert is logged with the agent’s own words, so a systematic misunderstanding shows up as a pattern rather than as noise. A call that changes *nothing* also counts as a failure: the record is the product, and an agent that explains a brilliant normalization in prose has produced nothing.

The four ways this measurement could still lie

failure	what stops it
inflated novelty	the ledger re-derives every “new” against the frozen baseline itself and reports disagreement with the record by name.
flattering aliases	calling two things the same operator hides a lost capability. The skeptic is aimed at this specifically, and a refinement must state what it breaks for existing users.
a threshold chosen after the fact	the differ writes metric and threshold into the record before running anything. A criterion no valid configuration could meet is exactly how the discovery loop lost a week.
a contaminated oracle	the reference lives in a separate interpreter. A Plexus process that could import it could borrow its answer, and a test that can be contaminated is not a test.

6. The phases

Seven stages. **I stop at the end of each one and wait for you.** That is the partition.

Goal. Make every later measurement trustworthy *before* taking one. Nothing here is science; each item makes a question askable.

What had to be done

1. Choose a target with running code, a licence we can build on, and biology worth having.
2. Freeze what Plexus can already say, before reading a line of the target.
3. Put the reference in its own interpreter, and forbid it in ours.
4. Prove the reference is deterministic at a fixed seed.
5. Produce the first reference artefact, and check it against its own figure.
6. Make every reference artefact carry the code and the versions that made it.
7. Refuse a reference run that hits an array bound instead of a biological limit.
8. Write the record down as *rules*, not as a template.
9. Self-test the rules against a record built to break every one of them.
10. Let the ledger overrule the record, and report the disagreement.
11. Self-test that overruling.
12. Extract the mechanism inventory mechanically, not from memory.
13. Add the mechanisms that are not classes and that a scan cannot see.
14. Make every agent call a guarded transaction.
15. Run one real agent call end to end before trusting the plumbing.
16. Make the paper itself readable to an agent.

Phase 0 — what actually happened, point for point

. state	the issue found, and what was interesting
1. done	Four candidates were compared. jax-morph won on a criterion I had not expected to matter: <i>its own architecture is an operator algebra</i> , so it tests the Plexus thesis directly rather than only supplying vocabulary.
2. done	52 registered contracts — 42 canonical, 10 aliases, 10 families. That set is the whole comparison.
3. done	The devcontainer reaches PyPI, which the Okuda corpus note had recorded as blocked (only <code>arxiv.org</code> and <code>github.com</code>). That single fact is what makes an executable oracle possible here at all.
4. done	Held. Two runs from one seed are bit-identical in position and in which cells are alive. Had it failed, every later difference would have been unattributable.
5. done	$4 \rightarrow 82$ cells in 40 macro-steps; radius of gyration $0.65 \rightarrow 3.23$. The figure shows a rounded, jammed cluster — the expected phenomenology, not a blown-up artefact.
6. done	Each run directory carries the script, the clone’s commit, the interpreter and the resolved package versions. A trajectory with no provenance is a number of unknown origin, and this campaign has already been burned by one.
7. done	Their division step exposes a <code>division_overflow</code> counter. The smoke run checks it and <i>stops</i> if it is non-zero — the same defect as our 1778-cell ceiling, in their units, caught before it can produce a conclusion.
8. done	Twelve rules. The hardest to write was not a type check: it is that <i>a verdict is an obligation</i> . “New” must survive the baseline; “alias” must name a registered contract and say what differs.
9. done	All twelve fire on a record built to break them, and a well-formed record passes in silence. The rules matter more than the catch — a validator whose failures cannot be enumerated is one nobody can argue with.
10. done	The ledger is not obliged to believe the record.
11. done	Verified in both directions, and the demotion is reported as a dispute rather than applied quietly.
12. done	20 classes, each with its file and line, its constructor parameters and its docstring. No LLM: an enumeration a human can check against <code>git grep class</code> in ten seconds.
13. done	The four most interesting mechanisms are not classes. The time-scale step taxonomy, the declared-dataflow composition contract, the explicit operator split and the stochastic trace contract live in prose and base classes. A purely automatic extraction — the obvious way to build an atlas at scale — would have missed precisely the entries most likely to be new. They were added by hand, and the record says they were added by hand.
14. done	Blast radius, rung, twelve rules; revert with the agent’s own words logged; two failures mark the mechanism <i>blocked</i> rather than skipping it. A call that changes nothing counts as a failure.
15. done	The excavator on <code>Division</code> , committed on the first attempt. See §7.
16. done	Found <i>by</i> item 15: the agent could not open the PDF (no <code>pdftotext</code> in this image), so it anchored its citations to the library’s guides and said so. PyMuPDF was already installed; the paper is now plain text with page markers, and <code>paper.py --grep</code> finds a term with its page number.

Phase 1 — Read all 24 mechanisms

DONE (36 min, 24 of 24)

The excavator on every mechanism: what it does to the state, its equations, the role of every knob, and a **surprises** list — the implicit units, orderings, guards and magic constants a reimplementer gets wrong. On the single call already made this took 6.3 minutes, so the phase is about two and a half hours if nothing needs a second attempt.

Stop condition: 24 entries at *inspected*, every code path resolving to a real line, zero blocked.

What I will be watching: whether each entry states what it could *not* determine. An unstated uncertainty becomes someone else's false belief, which is how the last campaign got into trouble — and the one call so far did this well, unprompted.

Phase 2 — Normalize, and let the skeptic attack

DONE (73 min)

Each mechanism becomes a typed Plexus contract with a verdict; the skeptic then tries to break each verdict; refuted ones go round again with the refutation in front of them.

The output is the first real ledger, and the first honest answer to “is the operator algebra converging?”

My prediction, on the record before we start: the five pairwise potentials will normalize to *one* existing contract with several interchangeable implementations; the genuinely new entries will be the three architectural ones, and those improve the language rather than the biological vocabulary. If that is wrong I want it to be wrong loudly, so it is written here.

Phase 3 — Implement what is missing

DONE (56 min + reconciliation)

Every *new* and every *refinement* becomes a Plexus operator in the anti-chamber, with a test of one property statable without the reference: a conservation law, a sign, a limit, a symmetry. Nothing enters the validated library at this stage.

Phase 4 — Differential validation

in flight

Metric and threshold first; then the same initial condition through both implementations; then subtract. **A failure here is the most informative outcome available** — it is the reproduction telling us the contract is wrong, which is the whole reason for choosing a target with code.

Phase 5 — The forward figure

IN PROGRESS (it runs)

Forward only, deliberately first. A composition of the normalized operators — growth, division, mechanics, secretion, diffusion — run in the Plexus engine and set beside the same composition run in theirs. No optimisation, no gradients: this figure exists to show that the *vocabulary* is faithful before anything is asked to learn through it.

It is also the honest order of work. If the forward figure disagrees with the reference, no amount of gradient descent on top will fix it — it will only hide the disagreement inside a fitted parameter.

Their headline result, and an expected feature of Plexus 2 rather than a bonus. Figure 5 of the paper is engineered morphogenesis: a small gene network is *optimised* until a growing cluster self-organises into a specified structure. Reproducing it exercises the inverse half of Plexus — gradients through the simulator itself.

We are not starting from zero here.

- `prototype/inverse_slime/` already does this shape of thing in Plexus: differentiable reimplementations of the field operators, validated against the engine to $\sim 10^{-7}$, and a teacher-forced one-step Adam fit that recovers all four differentiable slime parameters (mean error 0.001 against 0.283 for the black-box search — about $224\times$ better). It also records precisely where the approach stops: the hard `torch.where` branches in `sense` and `bounce`, which need a smooth surrogate.
- `ParticleGraph/src/ParticleGraph/models/graph_trainer.py` is the training loop that pattern came from.

What is genuinely new here, and worth being careful about. Slime recovers parameters from a trajectory; Figure 5 optimises parameters toward a *shape*, through a rollout that changes the number of cells. Division and death are discrete, so a pathwise gradient does not exist through them — which is exactly what the reference’s stochastic trace contract solves, and exactly why that contract is one of the mechanisms the atlas is normalizing in Phase 2. The inverse half of Plexus may end up needing it as a language feature, not just as a jax-morph detail. That would be the strongest possible result from this exercise.

Validated operators leave the anti-chamber with a library page and a spec that composes each one with an operator Plexus already had — a vocabulary item that only works beside its own siblings has not joined the language. Then the part that is not reproduction at all: run compositions the authors did not, and see what the new vocabulary does.

7. Phase 0 report — 31 July, 16:45

This is a phase boundary. All sixteen items are done; the sixteenth was created by the fifteenth. I have stopped here.

How it was checked. Every instrument was run, not inspected. The record validator and the saturation ledger each carry a self-test built to break them, and both were run to failure first: the validator against a record violating all twelve rules, the ledger against a record whose verdicts it is expected to overrule. The oracle was checked by the one property that makes it an oracle at all — determinism at a fixed seed — before any artefact of it was written down. And the agent layer was checked the only way that means anything: by running a real call, all the way through the driver, and reading what came out.

What was built

An oracle in its own interpreter. `oracle.py` builds a separate virtual environment, installs the reference from our local clone rather than from PyPI — so the record’s evidence points at bytes we can read — and records the clone’s commit, the interpreter and every resolved package version. Every run gets a directory containing the script that produced it and a copy of that provenance.

A baseline that cannot drift. `registry_view.py` enumerates the promoted registry into a frozen file. Saturation is measured against *that*, never against the live registry: a registry that grows during the run cannot measure its own growth.

A record that is a set of rules. `record.py` holds twelve, and a status ladder whose every rung is earned by an artefact rather than by an assertion — *inspected* needs a code path that resolves to a line that exists, *implemented* needs a module that registers the name it claims, *validated* needs an oracle run on disk with a threshold that was written before it.

A ledger that may overrule the record, and a driver that gives agents no authority (§5.1).

What the first real agent call found

The excavator was given one mechanism — their cell-division step — and 10 minutes. It used 6.3, committed on the first attempt, and produced an entry with the update equations, the role of every constructor argument, the declared *and* undeclared state it touches, and nine traps a reimplementer would fall into. Three are worth repeating here because they are the kind of thing that decides whether a reproduction works:

- **The volume-halving factor is dimension-dependent** ($2^{-1/d}$), so what looks like a shape parameter is really a physics parameter. Hard-code 1/2, or use the 3D factor in 2D, and volume conservation silently breaks.
- **The daughter is placed using the *new* radius**, so the two cells end up exactly touching. Using the pre-division radius overlaps or gaps them — a one-symbol error with a visible consequence.
- **The code contradicts its own docstring.** The docstring gives the clean hazard; the code additionally clips negative rates to zero, uses a straight-through surrogate for the gradient, guards two norms, and silently caps divisions at capacity. *The code is authoritative* — and this is precisely the class of finding we chose a target with code in order to make.

It also stated plainly what it could *not* establish, unprompted: that it could not open the paper PDF, so the source-versus-paper check is still open; and that it could not tell whether oriented division is ever exercised in the shipped examples. That habit is the single behaviour the last campaign most lacked.

Still not started — the known gaps

1. **Twenty-three mechanisms are unread**, and until Phase 1 runs, the paper’s own claims are unchecked against its code everywhere except in the division step.
2. **No Plexus-side spec exists for any jax-morph composition.** `config/atlas/` is empty; Phase 4 cannot start until a spec can express their initial condition exactly.
3. **The differential metric is unwritten, and it is not obvious.** Their macro-step is an operator split; ours is a schedule. Two integrators of the same equations diverge trajectory-wise while agreeing on everything that matters. The metric must measure the contract and not the integrator — a real decision, not a tuning.
4. **Two mechanisms have no torch equivalent yet:** their screened-diffusion solve and their neural-ODE controller. Either may be more work than the rest of the repository combined.
5. **The agent could not run `record.py` itself** — it is approval-gated in this image — so it checked the rules by hand and the driver validated afterwards. That is the design, but it means

an agent cannot iterate against the validator inside one call, which will cost extra round-trips in Phase 2.

8. Phases 1–3 report — 31 July, 22:30

Three phases, about three and a half hours of agent time, six agents at once. What follows is what came back, including the parts where the loop was wrong about itself.

Phase 1 — what 24 readings found

Twenty-four excavations, 36 minutes wall clock, nothing blocked. The entries carry the update equations, the role of every knob, the state each step really touches, and a **surprises** list. Three findings are worth your time:

- **Two mechanisms in the code are not in the paper at all.** **Death** — the words “death”, “apoptosis” and “necrosis” appear zero times in the paper, whose forward model is explicitly “division, growth, mechanical stress sensing, and morphogen excretion and detection”. And **ActiveBrownianDynamics2D**, self-propulsion with rotational diffusion, which the paper’s passive, noise-free methods never mention. The library ships capability the publication does not claim. *Recording that is only possible because we chose a target with code.*
- **The code generalises the paper where the paper was specific.** Their Methods define relaxation as *deterministic* gradient descent on a Morse potential. The source implements a Langevin step in which $kT=0$ is that gradient descent and $kT>0$ is the thermal regime one appendix experiment used. One operator, two papers’ worth of behaviour — and the unification is the code’s, not the text’s.
- **Division hides a dimension-dependent physics constant in what looks like a shape argument.** The volume-halving factor is $2^{-1/d}$, so `n_space_dim` silently sets how much a cell shrinks when it divides. A reimplementer who hard-codes 1/2 breaks volume conservation and will never see an error.

Phase 2 — the first ledger, and a prediction that was half wrong

verdict	n	what it means
new contract	8	<code>adhere</code> , <code>agitate</code> , <code>apoptose</code> , <code>mechanosense</code> , <code>morphogen</code> , <code>regulate</code> , <code>relax</code> , <code>reorient</code>
implementation	6	a second, third or fourth numerical realisation of one of those eight
alias	2	already in the language (both to <code>attraction_repulsion</code>)
refinement	2	<code>cell_divide</code> and <code>cell_grow</code> must widen to admit these
out of scope	6	no biology to extract: integration, plumbing, a null potential

I predicted, in writing, before the phase ran: the pairwise potentials would collapse to one contract with several implementations, and the genuinely new entries would be the three architectural ones.

Half right, and the wrong half is the interesting one. The potentials did collapse — four of the five are implementations of a single `adhere` contract. But every architectural entry came

back *out of scope*, not new: the operator split, the time-scale taxonomy, the dataflow contract and the stochastic trace are how their framework is built, not biology it models. The genuinely new vocabulary is biological instead — a cell regulating itself, dying, sensing its own mechanical load. I would have bet the other way.

The skeptic earned its place. It challenged 5 of 23 verdicts, and all five were in the same neighbourhood: the pairwise potentials, where the line between “same contract, different numerics” and “different contract” is genuinely hard. Two aliases became new contracts on the second pass; one new contract became a refinement.

Phase 3 — and what verification found afterwards

Sixteen operator modules. Then an independent sweep – import each in its own interpreter, check it registers what the record claims, compare its typed signature to the record, run its test, and import all sixteen *together*. The first sweep reported **2 of 16 clean**. After reconciliation it reports **16 of 16**. What lay between:

- **Two agents chose the same implementation label** (`connectionist`) for two different vector fields on the same contract. Invisible in isolation; the registry rejects it the moment both are imported into one process, which is why importing them all together is now part of the sweep.
- **Five modules declared they read position.** A Plexus set stores `pos`. A declared state block that does not exist is a signature nobody can ever check.
- **Two operators masked dead cells and did not declare reading alive** — a performed read that the type does not mention.
- **One refinement could not be built at all**, and the module says so instead of routing around it: the shipped `cell_grow` is *structural* (it advances a rest volume and returns nothing) while the `jax-morph` law is *lateral* (it returns a radius increment), and the registry rightly refuses two implementations of one contract that disagree about kind — kind is biology. It is parked as `grow_radius`. Promoting it means changing what `cell_grow` is, which breaks its existing users. *That is the language decision I flagged as open at the last boundary, arriving as a concrete case.*

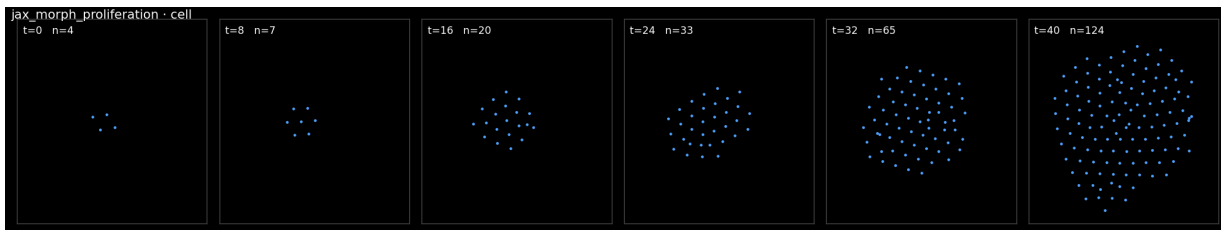
Where the loop was wrong about itself

Four defects in my own machinery, all found by running it, all fixed:

1. **The ledger counted rows, not contracts.** Four control steps normalize to one `regulate`; counting each as new vocabulary inflated the headline number from 8 to 11 — a 36% overstatement of exactly the quantity the atlas exists to measure. It now counts distinct contracts and reports repeats as *implementations*, which is the outcome `plexus2.tex` calls the good one.
2. **The reading agents had no Write tool.** Five calls ended by politely asking a human to approve a file write, in a headless subprocess with nobody listening. The retry recovered all five; a tool list that turns work into a question is still a bad tool list.
3. **The integrity check ate the work it was protecting.** I gave the driver an in-memory mirror of the record and told it to treat any deviation as tampering. With two driver processes running, each innocent commit looked like tampering to the other, and one of them “restored” a stale snapshot over a correct normalization. A record shared by several processes needs a lock in the filesystem, not a copy in one process’s memory.

4. **The skeptic’s refutations were being silently erased** by that same mirror — the loop’s one adversarial check, undone by its own integrity check. Recovered from the skeptic’s log and re-applied; all five disputes are on the record.

9. The forward figure, and the number that does not match



Plexus, running operators normalized out of their code: relax + grow + divide, their four founders, their parameters, 40 macro-steps. Compare with the reference strip in §4.

	Plexus	reference	
live cells, 4 →	124	82	51% high
radius of gyration	0.686 → 3.518	0.651 → 3.228	agrees to 9%
capacity used	124 of 600	82 of 140	neither run hit its buffer

The shape matches; the population does not. That is the honest state, and it is worth more than a figure that agreed. Two things are already ruled out: it is not an array bound (the buffer is five times the final count) and it is not an inert operator (every scheduled operator is recorded as having acted, on every frame it ran).

What it took to get even this far is itself a finding. The first run produced *zero* divisions in 40 frames, silently: `cell_divide` read the division rate as a Level buffer while the spec — and the growth operator standing next to it — kept it as a state block. So it read its own default of zero, and the run completed, and the movie looked plausible. Two operators from the same atlas disagreed about where a cell’s radius lives. Nothing raised. That is the class of defect this whole apparatus exists to catch, and it was caught by the acted ledger asking one question: *did this operator ever do anything?*

Phase 4 is now running that comparison properly for every mechanism, metric and threshold written down first.

10. Phase 6 is already possible — the gradient survives

You asked for Figure 5 by differentiability, and called it an expected feature of Plexus 2 rather than a bonus. So before Phase 4 finished I tested the one precondition nobody had: *does a gradient survive a Plexus rollout at all?*

It did not, for three reasons, and all three are now fixed or named:

1. **The engine discards every gradient by construction.** `engine.run` wraps its whole tick loop in `no_grad`. Sensible for forward generation, fatal for the inverse half. Lifting it is one opt-in argument, not a re-implementation — which matters, because `prototype/inverse_slime/` had to keep a *second* differentiable copy of the physics in step with the engine by hand. One physics, one tape, is worth having.

2. **A division event killed the tape.** Waking a daughter wrote the occupancy vector *in place*, and the growth operator multiplies its increment by that same vector — so autograd refused the multiply it had already recorded. The fix is the discipline JAX imposes on the reference for free and torch lets us skip: write a clone, publish the clone. A forward run cannot tell the two apart, which is exactly why nobody would have found this without asking for a gradient.
3. **A parameter gradient still does not flow**, because our tunables are Python floats baked in at construction. This is not a bug so much as an unmade decision, and the reference states the rule outright: *store as an array anything you want to learn*. That is a Plexus language change, and it is the next thing Phase 6 needs.

measured with <code>grad_probe.py</code>	result
gradient to the initial state, 6 frames	flows, $ \nabla = 1.19$
each operator alone (seed, relax, grow, divide)	all four flow
all four together, before the fix	tape broken at the first division
all four together, after the fix	flows, $ \nabla = 1.79$
20 frames through 23 real division events	flows, $ \nabla = 3.21$, $4 \rightarrow 27$ cells
gradient to an operator parameter	no path — parameters are floats

And here the two halves of the atlas meet. Phase 2 classified jax-morph’s four architectural mechanisms — the operator split, the time-scale taxonomy, the declared dataflow, the stochastic trace/replay/score contract — as *out of scope*, because none of them models any biology. That was the right call and it is only half the story: those four are precisely the infrastructure the *inverse* half of Plexus turns out to need. The trace contract in particular is how the reference gets a gradient through a discrete division at all, and it is the thing standing between us and a faithful Figure 5.

So the atlas’s first repository has produced, unprompted, a shortlist of what to add to the language — which is what Appendix E said it was for.

11. Decided, and what is still open

Decided (31 July). Run Phases 1–3 straight through, and stop for a joint review before any figure work. Then the figure in two steps: **forward only in Phase 5**, and **Figure 5 proper in Phase 6, through differentiability** — treated as an expected feature of Plexus 2 rather than an optional extra, building on `prototype/inverse_slime/` and the `ParticleGraph` trainer.

Still open, and not blocking.

1. **After this repository, which one?** A saturation curve only becomes an argument at the second and third framework. Cellular Potts (CompuCell3D) is the biggest step away from everything we have — cells as lattice domains rather than points — and therefore the sharpest test of whether the language is really complete.
2. **What happens to a refinement that breaks an existing user?** Widening a registered signature to admit a jax-morph mechanism can invalidate specs that already run. The record forces the cost to be stated; it does not say who pays it. I would rather agree that now than discover it during Phase 3.

Where things are. `atlas_record.yaml` — the record, the product. `campaign/analysis.md` — the append-only log. `_oracle/runs/` — reference artefacts with provenance. `_state/` — the frozen baseline, the extracted paper, the ledger, the reverts and the blocked list. `README.md` — how to run any of it.